[2CEIT401 – Operating Systems]

Practical: 9

AIM- Simulate various Disk Scheduling algorithms and different types of Page Replacement algorithms.

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥
U.V. Patel College of Engineering

Department of Computer Engineering

Aim: Simulate various Disk Scheduling algorithms and different types of Page Replacement algorithms.

1. Write a program to implement FIFO page Replacement algorithm.

Code:

```
#include <stdio.h>

int main() {
    int frames, pages, i, j;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int page[pages];
    printf("Enter page reference string: ");
    for (i = 0; i < pages; i++){
        scanf("%d", &page[i]);
    }
    int frame[frames];
    for (i = 0; i < frames; i++){
        frame[i] = -1;
    }
    int index = 0, faults = 0;

    for (i = 0; i < pages; i++) {
        int found = 0;

        for (j = 0; j < frames; j++) {
            if (frame[j] == page[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[index] = page[i];
            index = (index + 1) % frames;
            faults++;
        }
```

```
        // Display frames
        printf("Page %d -> ", page[i]);
        for (j = 0; j < frames; j++){
            printf("%d ", frame[j]);
        }
        printf("\n");
    }

    printf("Total Page Faults = %d\n", faults);
    return 0;
}
```

Output:

```
[24012011142@linuxserv prac9]$ ./que_1
Enter number of frames: 4
Enter number of pages: 5
Enter page reference string: 80 95 874 568 12
Page 80 → 80 -1 -1 -1
Page 95 → 80 95 -1 -1
Page 874 → 80 95 874 -1
Page 568 → 80 95 874 568
Page 12 → 12 95 874 568
Total Page Faults = 5
[24012011142@linuxserv prac9]$
```

2. Write a program to implement Optimal Page replacement algorithm.

Code:

```
#include <stdio.h>

int main() {
    int frames, pages, i, j, k;

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    int page[pages];
    printf("Enter page reference string: ");
    for (i = 0; i < pages; i++) {
        scanf("%d", &page[i]);
    }
    int frame[frames];
    for (i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    int faults = 0;

    for (i = 0; i < pages; i++) {
        int found = 0;

        // Check if page already in frame
        for (j = 0; j < frames; j++) {
            if (frame[j] == page[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            int pos = -1, farthest = i;
```

```
            for (j = 0; j < frames; j++) {
                int k;
                for (k = i + 1; k < pages; k++) {
                    if (frame[j] == page[k]) {
                        break;
                    }
                }

                if (k == pages) { // not used again
                    pos = j;
                    break;
                }

                if (k > farthest) {
                    farthest = k;
                    pos = j;
                }
            }

            if (pos == -1) {
                pos = 0;
            }

            frame[pos] = page[i];
            faults++;
        }

        // Display frames
        printf("Page %d -> ", page[i]);
        for (j = 0; j < frames; j++) {
            printf("%d ", frame[j]);
        }
        printf("\n");
    }

    printf("Total Page Faults = %d\n", faults);
    return 0;
}
```

Output:

```
[24012011142@linuxserv prac9]$ ./que_2
Enter number of frames: 4
Enter number of pages: 5
Enter page reference string: 80 95 874 568 12
Page 80 → 80 -1 -1 -1
Page 95 → 95 -1 -1 -1
Page 874 → 874 -1 -1 -1
Page 568 → 568 -1 -1 -1
Page 12 → 12 -1 -1 -1
Total Page Faults = 5
```

3. Write a program to implement FCFS Disk Scheduling algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, i;
    printf("Enter number of requests: ");
    scanf("%d", &n);

    int req[n];

    printf("Enter disk requests: ");
    for (i = 0; i < n; i++){
        scanf("%d", &req[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    int total_seek = 0;

    for (i = 0; i < n; i++) {
        int distance = abs(req[i] - head);
        total_seek += distance;

        printf("Move from %d to %d (Seek = %d)\n", head, req[i], distance);
        head = req[i];
    }
```

```
    printf("Total Seek Time = %d\n", total_seek);

    return 0;
}
```

Output:

```
[24012011142@linuxserv prac9]$ ./que_3
Enter number of requests: 6
Enter disk requests: 1 12 23 23 45 12
Enter initial head position: 1
Move from 1 to 1 (Seek = 0)
Move from 1 to 12 (Seek = 11)
Move from 12 to 23 (Seek = 11)
Move from 23 to 23 (Seek = 0)
Move from 23 to 45 (Seek = 22)
Move from 45 to 12 (Seek = 33)
Total Seek Time = 77
[24012011142@linuxserv prac9]$
```